Data Types

1. Fundamental Data Types

integer	float	complex	boolean	string
42	42.0	a+bj	True	"hello"

2. Advanced Data Types or Data Structure or Containers

List	Tuple	Set	Dictionary
[1,2,3]	(1,2,3)	{1,2,3}	{1:"audi",2:"Maruti"}

Int - Numeric value without any decimal

In [1]:
```
a=10
print(a)
```

```
10
```

In [2]:
```
# To check data type : use type() function
type(a)
```

Out[2]:
```
int
```

Float - Numeric value with decimal

In [3]:
```
b=3.0
print(b)
```

```
3.0
```

In [4]:
```
type(b)
```

Out[4]:
```
float
```

In [5]:
```
a=3
type(a)
```

Out[5]:
```
int
```

Type Casting: Convert from one data type to another data type

"Data Science & AI" by Siva Rama Krishna

In [6]:
```
b = 5.9
b
```
Out[6]: 5.9

In [7]:
```
type(b)
```
Out[7]: float

In [8]:
```
int(b)
```
Out[8]: 5

In [9]:
```
int(10.9999)
```
Out[9]: 10

In [10]:
```
float(int(5.4))
```
Out[10]: 5.0

Boolean - True/False

In [11]:
```
b1 = True
```

In [12]:
```
type(b1)
```
Out[12]: bool

In [13]:
```
type(False)
```
Out[13]: bool

In [14]:
```
int(True)
```
Out[14]: 1

In [15]:
```
int(False)
```
Out[15]: 0

In [16]:
```
bool(-1.0)
```
Out[16]: True

Complex

"Data Science & AI" by Siva Rama Krishna

Complex literals can be created by using the notation x + yj where x is the real component and y is the imaginary component.

In [17]:
```
x=10+20.5j
```

In [18]:
```
type(x)
```

Out[18]:
```
complex
```

In [19]:
```
# to get real value
x.real
```

Out[19]:
```
10.0
```

In [20]:
```
# to get imaginary value
x.imag
```

Out[20]:
```
20.5
```

In [21]:
```
int(10+5j)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Input In [21], in <cell line: 1>()
----> 1 int(10+5j)

TypeError: can't convert complex to int
```

In [22]:
```
complex(10.4)
```

Out[22]:
```
(10.4+0j)
```

"Data Science & AI" by Siva Rama Krishna

Strings

- A string in Python consists of a series or sequence of characters - letters, numbers, and special characters.
- We can use single quotes or double quotes to represent strings.

In [23]:
```
a='siva'
print(a)
```

```
siva
```

In [24]:
```
type(a)
```

Out[24]:
```
str
```

In [25]:
```
myString1 = 'Hello'
print(myString1)

myString2 = "Hello"
print(myString2)

myString3 = '''Hello'''
print(myString3)
```

```
Hello
Hello
Hello
```

- Multi-line strings can be denoted using triple quotes, ''' or """.

In [26]:
```
name='''siva
rama
krishna'''
```

In [27]:
```
name
```

Out[27]:
```
'siva \nrama \nkrishna'
```

In [28]:
```
var1 = 1234567
type(var1)
```

Out[28]:
```
int
```

In [29]:
```
var = "1234567"
type(var)
```

Out[29]:
```
str
```

In [30]:
```
float(var)
```

Out[30]:
```
1234567.0
```

In [31]:
```
a='si@1234'
```

In [32]:
```
int(a)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Input In [32], in <cell line: 1>()
----> 1 int(a)

ValueError: invalid literal for int() with base 10: 'si@1234'
```

Length of string

"Data Science & AI" by Siva Rama Krishna

In [33]:
```
myString = "Hello"
len(myString)
```

Out[33]: 5

Indexing (accessing characters in a string)

- Strings can be indexed - often synonymously called subscripted as well.
- The first character of a string has the index 0.

In [34]:
```
#print first Character
print(myString[0])
```

```
H
```

In [35]:
```
#last char d[len(d)-1] or d[-1]
print(myString[-1])
```

```
o
```

In [36]:
```
#print character using negative indexing
print(myString[-3])
```

```
l
```

In [37]:
```
# If we try to access index with out of the range, we will get index error.
print(myString[5])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Input In [37], in <cell line: 2>()
      1 # If we try to access index with out of the range, we will get index er
ror.
----> 2 print(myString[5])

IndexError: string index out of range
```

In [38]:
```
# If we try to access by using use decimal number, we will get errors.
print(myString[1.5])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Input In [38], in <cell line: 2>()
      1 # If we try to access by using use decimal number, we will get errors.
----> 2 print(myString[1.5])

TypeError: string indices must be integers
```

"Data Science & AI" by Siva Rama Krishna

Slicing the strings

[start index (inclusive) : end index (exclusive) : increment/decrement]

[default increment = +1]

In [39]:
```
#slicing 2nd to 5th character (start index to end-1 index)
myString[2:5]
```

Out[39]: 'llo'

In [40]:
```
myString[0:3]
```

Out[40]: 'Hel'

In [41]:
```
myString[:4]            # slicing (0 to end-1)
```

Out[41]: 'Hell'

In [42]:
```
myString[2:]            # slicing (Start index to all)
```

Out[42]: 'llo'

In [43]:
```
myString[:]            # slicing (Start to all)
```

Out[43]: 'Hello'

In [44]:
```
myString[0::2]
```

Out[44]: 'Hlo'

In [45]:
```
myString[4:0:-1]        # slicing with decrement
```

Out[45]: 'olle'

In [46]:
```
myString[2:45]         # slicing with increment
```

Out[46]: 'llo'

In [47]:
```
#print in reverse order
myString[::-1]
```

Out[47]: 'olleH'

String Concatenation

Joining of two or more strings into a single one is called concatenation.

The + operator does this in Python. Simply writing two string literals together also concatenates them.

"Data Science & AI" by Siva Rama Krishna

The * operator can be used to repeat the string for a given number of times.

In [48]:
```
#concatenation of 2 strings
'Hello' + " "+ "World"
```

Out[48]: 'Hello World'

In [49]:
```
"hello "+"world"
```

Out[49]: 'hello world'

In [50]:
```
"siva"+2
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Input In [50], in <cell line: 1>()
----> 1 "siva"+2

TypeError: can only concatenate str (not "int") to str
```

In [51]:
```
#repeat string n times
"AI" * 5
```

Out[51]: 'AIAIAIAIAI'

In [52]:
```
user = "satish"
lines = 100

print("Congratulations, " + user + " ! You just wrote " + str(lines) + " lin
```

```
Congratulations, satish ! You just wrote 100 lines of code
```

In [53]:
```
print("Congratulations, " ,user , " ! You just wrote " ,lines, " lines of co
```

```
Congratulations,  satish  ! You just wrote  100  lines of code
```

String Immutable

In [54]:
```
myString = "Hello"
myString[4]
```

Out[54]: 'o'

"Data Science & AI" by Siva Rama Krishna

In [55]:
```
# Strings are immutable. This means that elements of a string cannot be chan
# We can simply reassign different strings to the same name.

myString = "Hello"
myString[4]='s' # strings are immutable
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Input In [55], in <cell line: 5>()
      1 # Strings are immutable. This means that elements of a string cannot be
changed once it has been assigned.
      2 # We can simply reassign different strings to the same name.
      4 myString = "Hello"
----> 5 myString[4]='s'

TypeError: 'str' object does not support item assignment
```

String Methods

In [56]:
```
a="gooD moRNing"
```

capitalize --> Capatilise only first letter and remaning all to small --> Sentence case in MS.Word

In [57]:
```
a.capitalize()
```

Out[57]: 'Good morning'

title --> Capatilise first letter of each word and remaning all to small

In [58]:
```
a.title()
```

Out[58]: 'Good Morning'

upper --> convert complete string to upper case

In [59]:
```
a.upper()
```

Out[59]: 'GOOD MORNING'

lower --> convert complete string to lower case

In [60]:
```
a.lower()
```

Out[60]: 'good morning'

find --> To find a string in the input string

"Data Science & AI" by Siva Rama Krishna

In [61]:
```
a.find("mo")

# It returns the index position of string if available
# It returns -1 if not available
```

Out[61]: 5

count --> To count how many times the string available in the main string

In [62]:
```
a.count("o")
```

Out[62]: 3

split

In [63]:
```
st = 'my name is Sam'
st.split()                # Split with space (default)
```

Out[63]: ['my', 'name', 'is', 'Sam']

join

In [64]:
```
a= ['This', 'will', 'split', 'all', 'words', 'in', 'a', 'list']
' '.join(a)
```

Out[64]: 'This will split all words in a list'

replace

In [65]:
```
# To replace the old string  with new string
s1 = "Good morning"
print(s1)
id(s1)
```

```
Good morning
```

Out[65]: 1915774902192

In [66]:
```
s1 = s1.replace("Good", "Bad")
print(s1)
id(s1)
```

```
Bad morning
```

Out[66]: 1915775007024

"Data Science & AI" by Siva Rama Krishna

In [67]:

```python
my_str='Siva123'

print(my_str.isdigit()) #test if string contains only numbers

print(my_str.isalnum()) #check if it is alphanumeric

print(my_str.isalpha()) #check if all char in the string are alphabetic

print(my_str.istitle()) #test if string contains title words

print(my_str.isupper()) #test if string contains upper case

print(my_str.islower()) #test if string contains lower case

print(my_str.isspace()) #test if string contains spaces

print(my_str.endswith('s')) #test if string endswith a s

print(my_str.startswith('S')) #test if string startswith S
```

```
False
True
False
True
False
False
False
False
True
```

"Data Science & AI" by Siva Rama Krishna